CASE STUDY

OBJECT-ORIENTED PROGRAMMING IN C++

Bank Management System

Submitted by	Parnika Lodha
Programme	BCA Data Science
University	Chandigarh University
Section	25BCD-2A
UID	25BCD10078
Subject	Object-Oriented Programming (C++)
Submitted to	Mr. Jitendra Sir

C++ • Classes and Objects • Four Pillars of OOP

1. Introduction

Object-Oriented Programming (OOP) is a programming approach that organizes a program around classes and objects. It helps make software modular, reusable, and easier to maintain. C++ supports important OOP features such as encapsulation, inheritance, polymorphism, and abstraction.

This case study presents a simple console-based Bank Management System in C++. It demonstrates how account details and banking operations can be organized using classes and objects.

2. Problem Statement

Manual management of bank accounts can be time-consuming and may lead to calculation errors. A basic computer program can represent account holders, store account balances during execution, and perform deposit and withdrawal operations with validation.

The purpose of this project is to implement a simple Bank Management System using C++ OOP concepts. It supports savings and current account types and displays account details and updated balances.

3. Objectives

• Understand classes, objects, and constructors in C++.

• Demonstrate encapsulation, inheritance, polymorphism, and abstraction.

• Perform deposit and withdrawal operations with basic validation.

• Use header files and preprocessor directives appropriately.

• Create a program that is easy to read, test, and extend.

4. Software and Hardware Requirements

Item	Requirement
Programming language	C++ (C++17 compatible)
Compiler	g++ or another C++17-compatible compiler
IDE / Editor	Visual Studio Code
Libraries	Standard headers: iostream, string, iomanip
Hardware	Computer capable of running VS Code and a C++ compiler

5. System Design and Class Relationship

The abstract base class BankAccount contains common account data and operations. SavingsAccount and CurrentAccount inherit from it and provide their own account type. The balance is private, and public member functions control deposits, withdrawals, and display.

BankAccount (Abstract Base Class) accountHolder • private balance deposit() • withdraw() • displayDetails() • getAccountType()
Inheritance ↓
SavingsAccount **CurrentAccount** getAccountType() returns Savings Account getAccountType() returns Current Account

6. Complete C++ Source Code

```
/*
    PROJECT: BANK MANAGEMENT SYSTEM
    LANGUAGE: C++
    OOP: Encapsulation, Inheritance, Polymorphism, Abstraction
*/

#include <iostream>
#include <string>
#include <iomanip>
using namespace std;

class BankAccount
{
protected:
    string accountHolder;

private:
    double balance;

public:
    BankAccount(string name, double initialBalance)
    {
        accountHolder = name;
        if (initialBalance >= 0)
            balance = initialBalance;
        else
            balance = 0;
    }

    virtual ~BankAccount() = default;

    void deposit(double amount)
    {
        if (amount > 0)
        {
            balance += amount;
            cout << "Rs. " << fixed << setprecision(2)
                 << amount << " deposited successfully.\n";
        }
        else
            cout << "Invalid deposit amount.\n";
    }

    void withdraw(double amount)
    {
        if (amount <= 0)
            cout << "Invalid withdrawal amount.\n";
        else if (amount > balance)
            cout << "Insufficient balance.\n";
        else
        {
            balance -= amount;
            cout << "Rs. " << fixed << setprecision(2)
                 << amount << " withdrawn successfully.\n";
        }
    }

    void displayDetails() const
    {
        cout << "Account Holder: " << accountHolder << endl;
        cout << "Account Type: " << getAccountType() << endl;
        cout << "Balance: Rs. " << fixed << setprecision(2)
             << balance << endl;
    }

    virtual string getAccountType() const = 0;
};

class SavingsAccount : public BankAccount
{
public:
    SavingsAccount(string name, double balance)
        : BankAccount(name, balance) {}

    string getAccountType() const override
    {
        return "Savings Account";
    }
};

class CurrentAccount : public BankAccount
{
public:
    CurrentAccount(string name, double balance)
        : BankAccount(name, balance) {}

    string getAccountType() const override
```

```
    {
        return "Current Account";
    }
};

int main()
{
    cout << "====================================\n";
    cout << "      BANK MANAGEMENT SYSTEM\n";
    cout << "====================================\n\n";

    SavingsAccount account1("Parnika", 5000);
    CurrentAccount account2("Rahul", 10000);

    cout << "--- SAVINGS ACCOUNT ---\n";
    account1.displayDetails();

    cout << "\nDepositing Rs. 2000...\n";
    account1.deposit(2000);

    cout << "\nWithdrawing Rs. 1000...\n";
    account1.withdraw(1000);

    cout << "\nUpdated Savings Account Details:\n";
    account1.displayDetails();

    cout << "\n--- CURRENT ACCOUNT ---\n";
    account2.displayDetails();

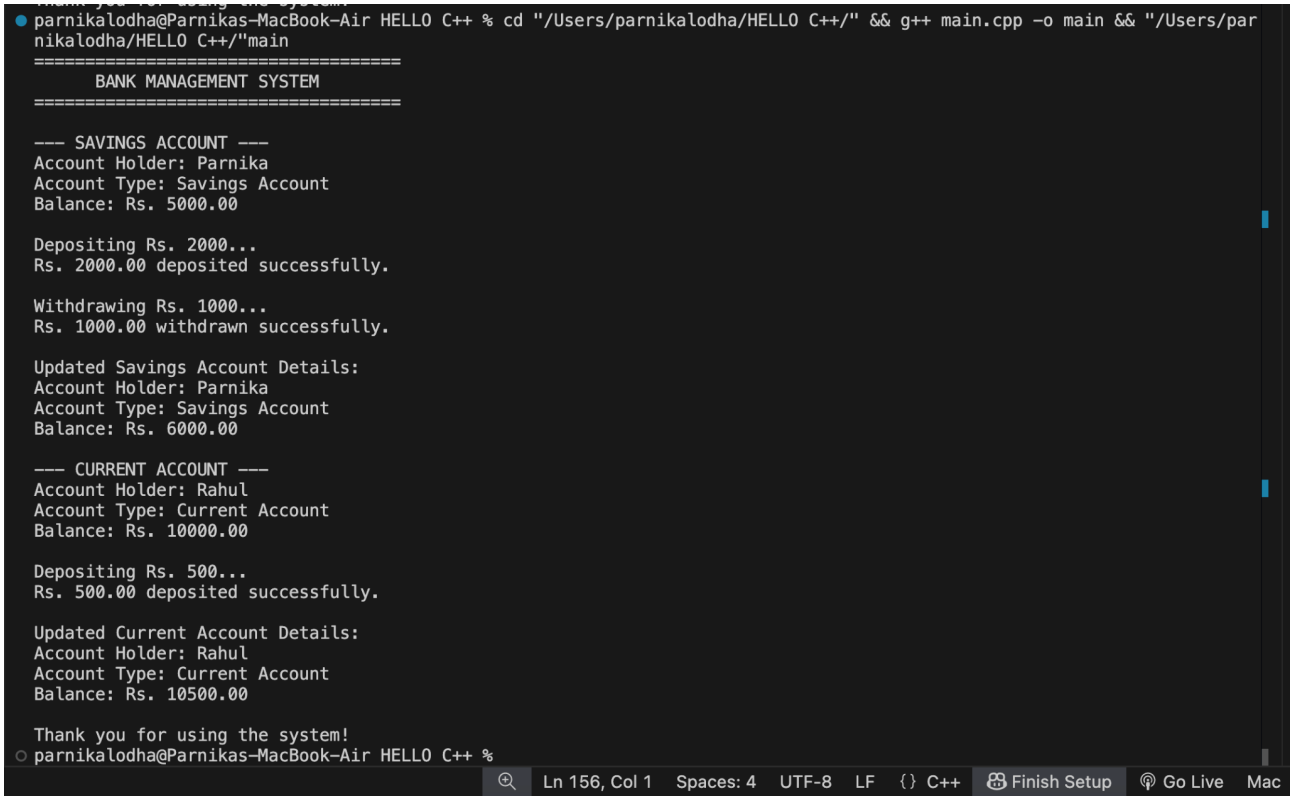
    cout << "\nDepositing Rs. 500...\n";
    account2.deposit(500);

    cout << "\nUpdated Current Account Details:\n";
    account2.displayDetails();

    cout << "\nThank you for using the system!\n";
    return 0;
}
```

7. Program Output

The following screenshot was captured from the program run in Visual Studio Code. It shows the savings and current account operations and their updated balances.



8. Explanation of OOP Concepts

A. Encapsulation

The data member balance is declared private. This prevents ordinary direct access to it from outside the class. Public functions such as deposit(), withdraw(), and displayDetails() provide controlled access to the account's operations.

B. Inheritance

SavingsAccount and CurrentAccount inherit publicly from BankAccount. They reuse its constructor and common functions instead of repeating the same code.

C. Polymorphism

The function getAccountType() is declared virtual in the base class and overridden in both derived classes. The function returns a different account type depending on the object.

D. Abstraction

The pure virtual function getAccountType() = 0 makes BankAccount an abstract class. It defines what derived classes must provide without specifying the account type for them.

E. Constructor

The BankAccount constructor initializes the account holder and starting balance. The derived-class constructors call it through an initializer list.

F. Header Files and Preprocessor Directives

The #include directives include the standard C++ headers. iostream supports input and output, string supports text values, and iomanip formats the displayed balance. These are standard library headers; no third-party libraries are required.

9. Advantages

• Reusable code through inheritance.

• Clear organization of account data and operations.

• Basic validation for deposits and withdrawals.

• Easy extension with more account types and features.

• Demonstrates key OOP concepts in one practical example.

10. Limitations

• Account data is not saved permanently after the program closes.

• The system has no database, login system, or transaction history.

• It does not handle all financial precision and security requirements for real banking.

11. Real-World Applications

OOP is used in banking software, hospital management, student information systems, railway reservation systems, and e-commerce applications. Classes can represent customers, accounts, transactions, products, or bookings.

12. Conclusion

This case study demonstrates a Bank Management System implemented in C++ using classes and objects. It applies encapsulation, inheritance, polymorphism, and abstraction to organize account data and banking operations. The program compiles with a C++17-compatible compiler and provides a foundation for future improvements such as file handling, database integration, and authentication.